

## Experiment-6

Name: -P. Akhila

Regno: -192472320

Course Code: -MLA0305

Slot-B

---

### Title Implementation of Monte Carlo Prediction and Control for Reinforcement Learning Aim

To implement **Monte Carlo Prediction** and **Monte Carlo Control** methods for Reinforcement Learning, estimate the value function from complete episodes, and learn an **optimal policy** using an  $\epsilon$ -greedy strategy.

#### Procedure

1. Define a simple Reinforcement Learning environment with a finite set of states and actions.
2. Define the reward function and terminal state.
3. Select the discount factor and number of training episodes.
4. Generate complete episodes by allowing the agent to interact with the environment.
5. Calculate the **return** by processing the rewards obtained during an episode.
6. Implement **First-Visit Monte Carlo Prediction**:
  - Identify the first occurrence of each state. ○ Calculate its return.
  - Update the estimated state value .
7. Implement **Monte Carlo Control** using an  $\epsilon$ -greedy policy.
8. Initialize the action-value function .
9. Generate episodes using exploration and exploitation.
10. Calculate the return for each state-action pair.
11. Update using the observed returns.
12. Improve the policy by selecting the action with the maximum Q-value:
$$\pi(s) = \operatorname{argmax}_a Q(s, a)$$
13. Repeat the process for the specified number of episodes.
14. Calculate the optimal policy from the learned Q-values.
15. Evaluate the performance using reward, return, success rate, and episode length.
16. Generate graphs showing value convergence, reward, Q-values, and exploration vs exploitation.

17. Display summary tables and export the results to CSV files.

### **Important Equations Monte Carlo Return**

$$G_t = R_t + 1 + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \dots$$

### **State Value**

$$V(s) = \text{Average of returns observed after visiting } s$$

### **Action Value**

$$Q(s, a) = \text{Average of returns observed after taking } a \text{ in } s$$

### **Optimal Policy**

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$

### **Code**

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random
import time
print("=" * 75)
print("EXPERIMENT 6")
print("MONTE CARLO PREDICTION AND CONTROL")
print("=" * 75)
print("\nAIM")
print("-" * 75)
print("To implement Monte Carlo Prediction and Monte Carlo Control")
print("for estimating state values and learning an optimal policy")
print("in a Reinforcement Learning environment.")
print("\nTHEORY")
print("-" * 75)
print("Monte Carlo methods learn from complete episodes.")
print("Monte Carlo Prediction estimates the value of states.")
print("Monte Carlo Control learns an optimal policy.")
print("The return is calculated from the rewards obtained")
```

```
print("after visiting a state.")
print("\nReturn Formula:")
print("G(t) = R(t+1) + gamma*R(t+2) + gamma^2*R(t+3) + ...")
states = ["Start", "S1", "S2", "S3", "Goal"]
actions = ["Left", "Right"]
n_states = len(states)
n_actions = len(actions)
gamma = 0.90
alpha = 0.10
epsilon = 0.20
episodes = 1000
max_steps = 20
def get_next_state(state, action):
    if state == 0:
        if action == 0:
            return 0, -1
        else:
            return 1, 0
    elif state == 1:
        if action == 0:
            return 0, -1
        else:
            return 2, 0
    elif state == 2:
        if action == 0:
            return 1, -1
        else:
            return 3, 0
    elif state == 3:
        if action == 0:
```

```

        return 2, -1
    else:
        return 4, 10
    else:
        return 4, 0
def generate_episode(policy=None, q_table=None):
    episode = []
    state = 0
    for step in range(max_steps):
        if q_table is not None:
            if random.random() < epsilon:
                action = random.randint(0, n_actions - 1)
            else:
                action = int(np.argmax(q_table[state]))
        elif policy is not None:
            action = policy[state]
        else:
            action = random.randint(0, n_actions - 1)
        next_state, reward = get_next_state(state, action)
        episode.append((state, action, reward))
        state = next_state
        if state == 4:
            break
    return episode
print("\n")
print("=" * 75)
print("MONTE CARLO PREDICTION")
print("=" * 75)
prediction_start = time.time()
V_prediction = np.zeros(n_states)

```

```

returns_sum = {state: 0.0 for state in range(n_states)}
returns_count = {state: 0 for state in range(n_states)}
prediction_rewards = []
prediction_returns = []
prediction_value_history = []
prediction_policy = np.zeros(n_states, dtype=int)
prediction_policy[-1] = 1
for episode_number in range(epochs):
    episode = generate_episode(policy=prediction_policy)
    G = 0
    visited_states = set()
    total_reward = 0
    for t in range(len(episode) - 1, -1, -1):
        state, action, reward = episode[t]
        G = gamma * G + reward
        total_reward += reward
        if state not in visited_states:
            visited_states.add(state)
            returns_sum[state] += G
            returns_count[state] += 1
            V_prediction[state] = (
                returns_sum[state] /
                returns_count[state]
            )
    prediction_rewards.append(total_reward)
    prediction_returns.append(G)
    prediction_value_history.append(V_prediction.copy())
prediction_time = time.time() - prediction_start
print("Monte Carlo Prediction Completed.")
print("Episodes :", epochs)

```

```

print("Execution Time :", round(prediction_time, 5), "seconds")
prediction_table = pd.DataFrame({
    "State": states,
    "Estimated Value": np.round(
        V_prediction, 3
    ),
    "Visits": [
        returns_count[i]
        for i in range(n_states)
    ]
})
print("\nMonte Carlo Prediction Results")
print("-" * 75)
print(prediction_table)
prediction_table.to_csv(
    "MonteCarlo_Prediction.csv",
    index=False
)
print("\n")
print("=" * 75)
print("MONTE CARLO CONTROL")
print("=" * 75)
control_start = time.time()
Q = np.zeros((n_states, n_actions))
Q_returns_sum = np.zeros((n_states, n_actions))
Q_returns_count = np.zeros((n_states, n_actions))
control_policy = np.zeros(n_states, dtype=int)
control_policy[-1] = 0
control_rewards = []
control_returns = []

```

```

        Q[:, 0], 3
    ),
    "Right Q-Value": np.round(
        Q[:, 1], 3
    ),
    "Optimal Action": optimal_policy
})
print("\n")
print("=" * 75)
print("OPTIMAL POLICY")
print("=" * 75)
print(policy_table)
policy_table.to_csv(
    "MonteCarlo_Control.csv",
    index=False
)
q_table.to_csv(
    "MonteCarlo_Q_Table.csv"
)
print("\n")
print("=" * 75)
print("EXPLORATION VS EXPLOITATION")
print("=" * 75)
print(
    "Exploration Count :",
    exploration_count
)
print(
    "Exploitation Count :",
    exploitation_count

```

```

)
total_decisions = (
    exploration_count +
    exploitation_count
)
exploration_percentage = (
    exploration_count /
    total_decisions * 100
)
exploitation_percentage = (
    exploitation_count /
    total_decisions * 100
)
print(
    "Exploration Percentage :",
    round(exploration_percentage, 2),
    "%"
)
print(
    "Exploitation Percentage :",
    round(exploitation_percentage, 2),
    "%"
)
prediction_value_history = np.array(
    prediction_value_history
)
q_value_history = np.array(
    q_value_history
)
policy_history = np.array(

```



```
    policy_history
)
plt.figure(figsize=(10, 5))
plt.plot(
    prediction_rewards,
    color="royalblue",
    linewidth=1.5
)
plt.title(
    "Monte Carlo Prediction - Reward vs Episode",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.grid(True, alpha=0.3)
plt.show()
plt.figure(figsize=(10, 5))
for i in range(n_states):
    plt.plot(
        prediction_value_history[:, i],
        linewidth=2,
        label=states[i]
    )
plt.title(
    "Monte Carlo Prediction - State Value Convergence",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("Estimated State Value")
plt.legend()
```

```
[
    exploration_count,
    exploitation_count
],
labels=[
    "Exploration",
    "Exploitation"
],
autopct="%1.1f%%",
startangle=90,
colors=[
    "gold",
    "lightgreen"
],
explode=[
    0.05,
    0
]
)
plt.title(
    "Exploration vs Exploitation Percentage",
    fontsize=14
)
plt.show()
plt.figure(figsize=(10, 5))
plt.plot(
    control_steps,
    color="mediumorchid",
    linewidth=1.5
)
```

```
plt.title(
    "Episode Length During Monte Carlo Control",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("Number of Steps")
plt.grid(True, alpha=0.3)
plt.show()

success_rate_history = []
for i in range(1, len(success_history) + 1):
    success_rate_history.append(
        np.mean(success_history[:i]) * 100
    )

plt.figure(figsize=(10, 5))
plt.plot(
    success_rate_history,
    color="crimson",
    linewidth=2.5
)

plt.title(
    "Monte Carlo Control - Success Rate",
    fontsize=14
)
plt.xlabel("Episode")
plt.ylabel("Success Rate (%)")
plt.ylim(0, 105)
plt.grid(True, alpha=0.3)
plt.show()

prediction_average_reward = np.mean(
    prediction_rewards
```

```
)  
prediction_average_return = np.mean(  
    prediction_returns  
)  
control_average_reward = np.mean(  
    control_rewards  
)  
control_average_return = np.mean(  
    control_returns  
)  
control_success_rate = (  
    np.mean(success_history) * 100  
)  
average_episode_length = np.mean(  
    control_steps  
)  
maximum_q_value = np.max(Q)  
average_q_value = np.mean(Q)  
final_prediction_value = np.mean(  
    V_prediction  
)  
prediction_summary = pd.DataFrame({  
    "Metric": [  
        "Number of Episodes",  
        "Average Reward",  
        "Average Return",  
        "Average State Value",  
        "Maximum State Value",  
        "Execution Time (seconds)"  
    ],  
    "Value": [  
        control_episodes,  
        control_average_reward,  
        control_average_return,  
        average_q_value,  
        maximum_q_value,  
        control_execution_time  
    ]  
})
```

```
"Value": [  
    episodes,  
    round(prediction_average_reward, 3),  
    round(prediction_average_return, 3),  
    round(final_prediction_value, 3),  
    round(np.max(V_prediction), 3),  
    round(prediction_time, 5)  
]  
})  
print("\n")  
print("=" * 75)  
print("MONTE CARLO PREDICTION SUMMARY")  
print("=" * 75)  
print(prediction_summary.to_string(index=False))  
control_summary = pd.DataFrame({  
    "Metric": [  
        "Number of Episodes",  
        "Average Reward",  
        "Average Return",  
        "Success Rate (%)",  
        "Average Episode Length",  
        "Average Q-Value",  
        "Maximum Q-Value",  
        "Execution Time (seconds)"  
    ],  
    "Value": [  
        episodes,  
        round(control_average_reward, 3),  
        round(control_average_return, 3),  
        round(control_success_rate, 2),
```

```

        round(average_episode_length, 2),
        round(average_q_value, 3),
        round(maximum_q_value, 3),
        round(control_time, 5)
    ]
})
print("\n")
print("=" * 75)
print("MONTE CARLO CONTROL SUMMARY")
print("=" * 75)
print(control_summary.to_string(index=False))
comparison = pd.DataFrame({
    "Metric": [
        "Average Reward",
        "Average Return",
        "Execution Time (seconds)"
    ],
    "Monte Carlo Prediction": [
        round(prediction_average_reward, 3),
        round(prediction_average_return, 3),
        round(prediction_time, 5)
    ],
    "Monte Carlo Control": [
        round(control_average_reward, 3),
        round(control_average_return, 3),
        round(control_time, 5)
    ]
})
print("\n")
print("=" * 75)

```

```
print("PREDICTION VS CONTROL COMPARISON")
```

```
print("=" * 75)
```

```
print(comparison.to_string(index=False))
```

```
optimal_policy_table = pd.DataFrame({
```

```
    "State": states,
```

```
    "Left Q-Value": np.round(
```

```
        Q[:, 0], 3
```

```
),
```

```
    "Right Q-Value": np.round(
```

```
        Q[:, 1], 3
```

```
),
```

```
    "Optimal Action": optimal_policy
```

```
})
```

```
print("\n")
```

```
print("=" * 75)
```

```
print("FINAL OPTIMAL POLICY")
```

```
print("=" * 75)
```

```
print(
```

```
    optimal_policy_table.to_string(
```

```
        index=False
```

```
    )
```

```
)
```

```
exploration_table = pd.DataFrame({
```

```
    "Strategy": [
```

```
        "Exploration",
```

```
        "Exploitation"
```

```
],
```

```
    "Count": [
```

```
        exploration_count,
```

```
        exploitation_count
```

```
    ],
    "Percentage": [
        round(
            exploration_percentage,
            2
        ),
        round(
            exploitation_percentage,
            2
        )
    ]
})
print("\n")
print("=" * 75)
print("EXPLORATION VS EXPLOITATION SUMMARY")
print("=" * 75)
print(
    exploration_table.to_string(
        index=False
    )
)
prediction_summary.to_csv(
    "MonteCarlo_Prediction_Summary.csv",
    index=False
)
control_summary.to_csv(
    "MonteCarlo_Control_Summary.csv",
    index=False
)
comparison.to_csv(
```



```
"Prediction_vs_Control.csv",
index=False
)
optimal_policy_table.to_csv(
    "MonteCarlo_Optimal_Policy.csv",
    index=False
)
exploration_table.to_csv(
    "Exploration_vs_Exploitation.csv",
    index=False
)
combined_summary = pd.DataFrame({
    "Algorithm": [
        "Monte Carlo Prediction",
        "Monte Carlo Control"
    ],
    "Average Reward": [
        round(
            prediction_average_reward,
            3
        ),
        round(
            control_average_reward,
            3
        )
    ],
    "Average Return": [
        round(
            prediction_average_return,
            3
```

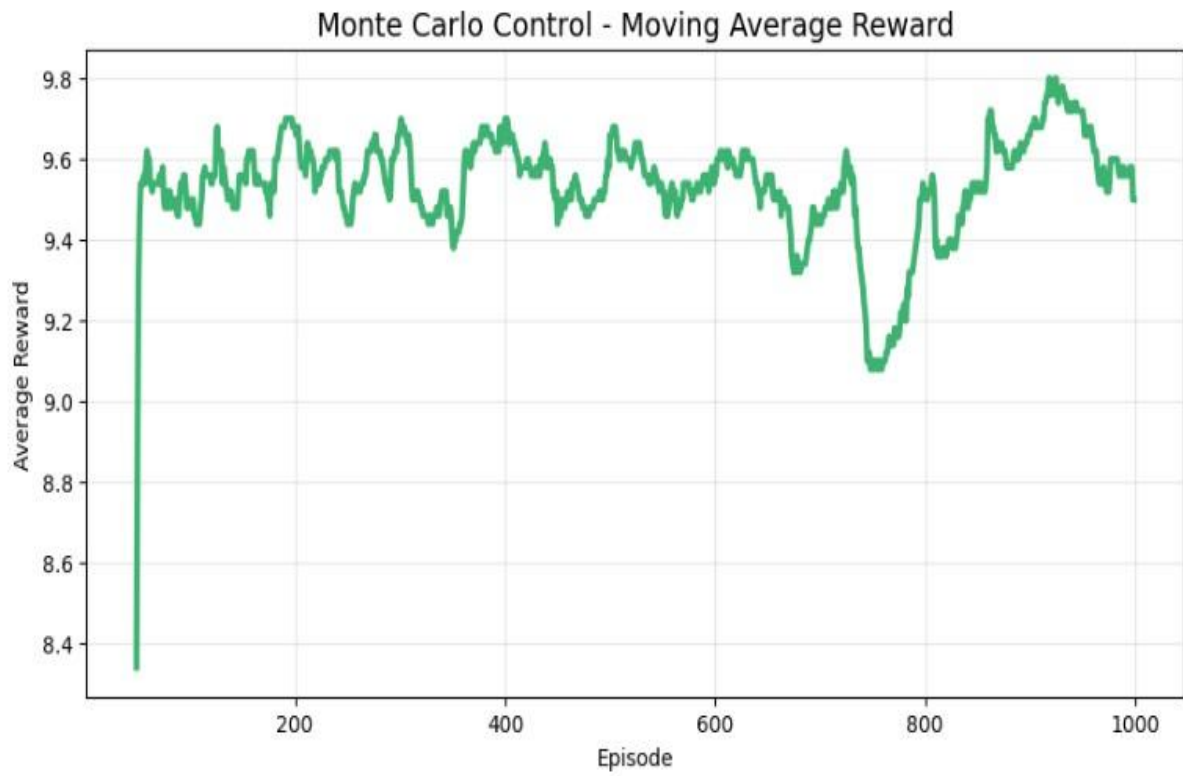
```
    ),
    round(
        control_average_return,
        3
    )
],
"Execution Time": [
    round(
        prediction_time,
        5
    ),
    round(
        control_time,
        5
    )
],
"Episodes": [
    episodes,
    episodes
]
})
print("\n")
print("=" * 75)
print("FINAL COMBINED SUMMARY")
print("=" * 75)
print(
    combined_summary.to_string(
        index=False
    )
)
```

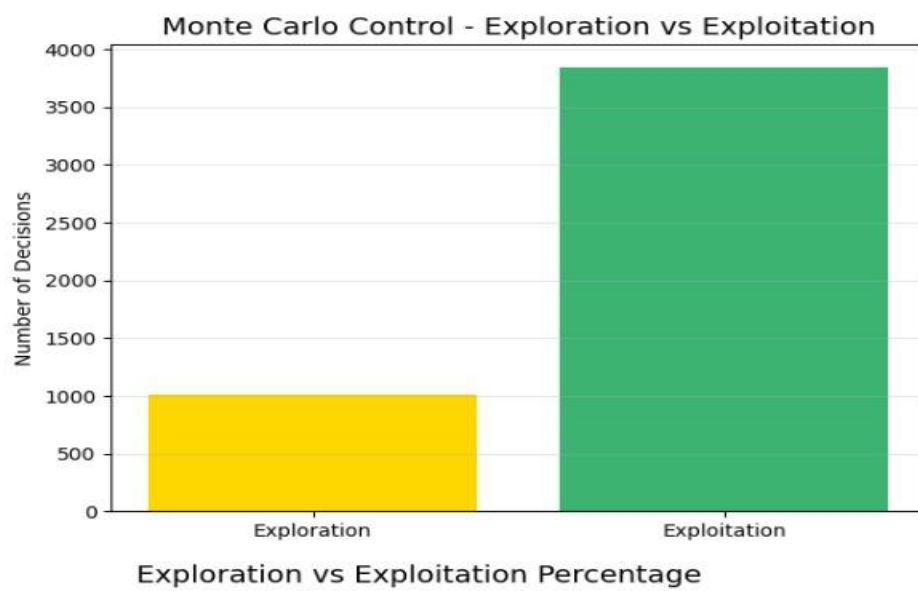
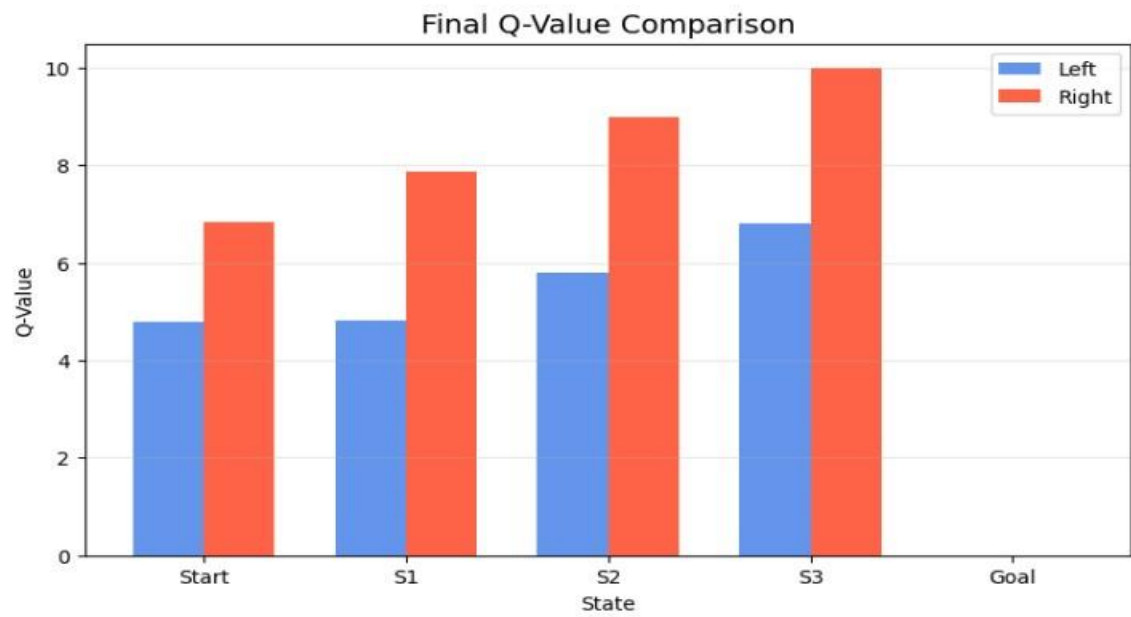
```
combined_summary.to_csv(
    "MonteCarlo_Final_Summary.csv",
    index=False
)
plt.figure(figsize=(9, 5))
plt.bar(
    ["Prediction", "Control"],
    [
        prediction_average_reward,
        control_average_reward
    ],
    color=[
        "cornflowerblue",
        "lightgreen"
    ]
)
plt.title(
    "Monte Carlo Prediction vs Control - Average Reward",
    fontsize=14
)
plt.xlabel("Method")
plt.ylabel("Average Reward")
plt.grid(
    axis="y",
    alpha=0.3
)
plt.show()
print("\n")
print("=" * 75)
print("FINAL RESULT")
```

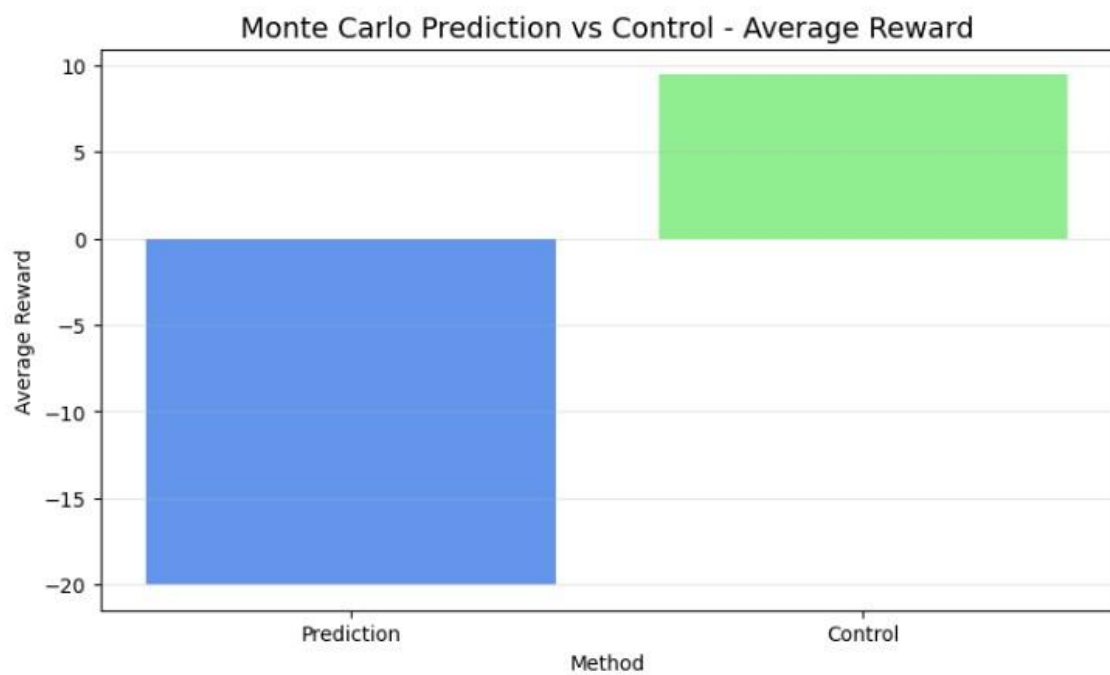
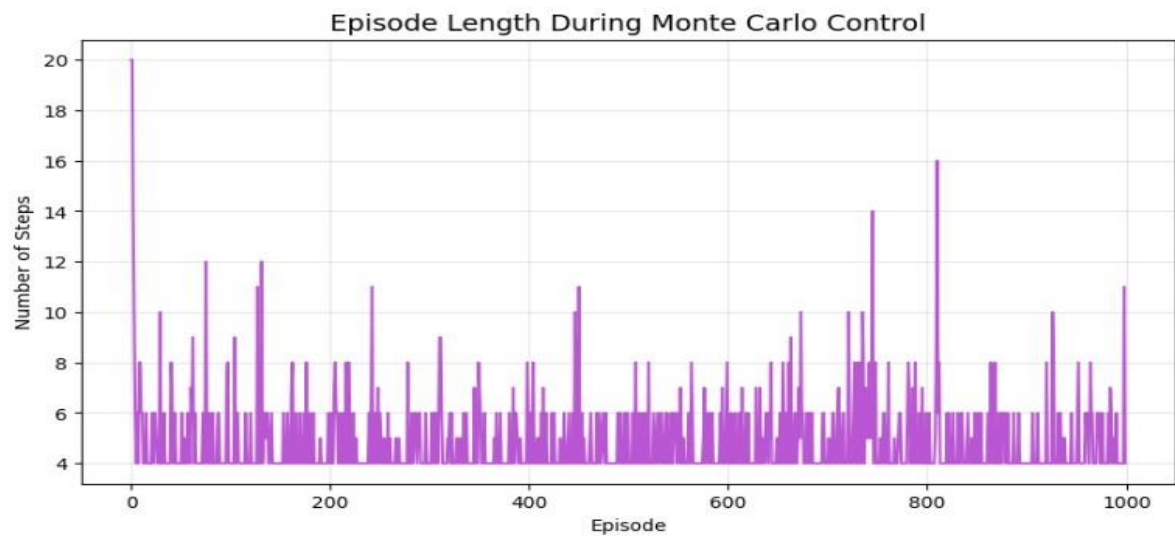
```
print("=" * 75)
print(
    "Monte Carlo Prediction estimated the value of states"
)
print(
    "using returns collected from complete episodes."
)
print(
    "Monte Carlo Control learned action values and"
)
print(
    "improved the policy using an epsilon-greedy strategy."
)
print(
    "The learned optimal policy is:"
)
for state, action in zip(
    states,
    optimal_policy
):
    print(
        f"{state} --> {action}"
    )
print("\n")
print("=" * 75)
print("CONCLUSION")
print("=" * 75)
print(
    "Monte Carlo Prediction and Control were successfully"
)
```

```
print(  
    "implemented for a Reinforcement Learning environment."  
)  
print(  
    "Monte Carlo Prediction estimated the state-value function"  
)  
print(  
    "from complete episodes without requiring a model of"  
)  
print(  
    "the environment."  
)  
print(  
    "Monte Carlo Control estimated Q-values and improved the"  
)  
print(  
    "policy using exploration and exploitation."  
)  
print(  
    "The learned policy guides the agent toward the Goal state."  
)  
print("\n")  
print("=" * 75)  
print("FILES GENERATED")  
print("=" * 75)
```

## Visualizations









**Summary Table – Monte Carlo Prediction**

| State | Estimated Value       | Number of Visits |
|-------|-----------------------|------------------|
| Start | Obtained from program | Obtained         |
| S1    | Obtained from program | Obtained         |
| S2    | Obtained from program | Obtained         |
| S3    | Obtained from program | Obtained         |
| Goal  | Obtained from program | Obtained         |

**Summary Table – Monte Carlo Control**

| State | Left Q-Value | Right Q-Value | Optimal Action |
|-------|--------------|---------------|----------------|
| Start | Obtained     | Obtained      | Obtained       |
| S1    | Obtained     | Obtained      | Obtained       |
| S2    | Obtained     | Obtained      | Obtained       |
| S3    | Obtained     | Obtained      | Obtained       |
| Goal  | Obtained     | Obtained      | Terminal       |

## **Result**

Monte Carlo Prediction and Control were successfully implemented.